

接入文档-HarmonyOS

喜马拉雅生态SDK HarmonyOS集成指南

概述

喜马拉雅生态SDK (XMEcoSDK) 为第三方应用提供音频内容服务，主要能力包括：音频播放控制、Web 组件内容接入、用户认证、悬浮播放器、锁屏播控、数据获取等。

系统要求

- HarmonyOS NEXT 5.1.1+ (API Level 19+)
- DevEco Studio: 需支持 5.1.1(19) SDK

1. 集成步骤

1.1 添加依赖

[复制代码](#)

```
// entry/oh-package.json5
{ "dependencies": { "@xmly/xmecosdk": "file:libs/xmecosdk.har" } }
```

执行 `ohpm install`。

1.2 权限配置

[复制代码](#)

```
// module.json5
"requestPermissions": [
  { "name": "ohos.permission.INTERNET" },
  { "name": "ohos.permission.GET_NETWORK_INFO" }
]
```

1.3 注册页面（悬浮窗必需）

唯一必须注册的 SDK 相关页面是 `pages/FloatingPlayer`（原因见 1.4）。接入方自己的业务页面照常注册，与 SDK 无关：

复制代码

```
{ "src": [ "pages/Index", "pages/FloatingPlayer" ] }
```

SDK 不提供也不要求 WebView 容器页面。接入方用自己的页面承载 H5（参考 demo 工程的 `WebViewDemo`，仅作接入示例，非必须）。

1.4 悬浮窗薄壳页面（必需）

这是 HarmonyOS 的平台限制，非可选：HAR 不能声明 page，悬浮窗所需的 `pages/FloatingPlayer` 页面必须由宿主在 `main_pages.json` 注册，否则悬浮窗无法弹出。悬浮窗的全部逻辑（拖拽/吸附/折叠/播控）都在 HAR 内，宿主只需提供这个 10 行薄壳页承载组件：

复制代码

```
import { FloatingPlayerComponent } from '@xmly/xmecosdk'

@Entry @Component
struct FloatingPlayer {
    build() { Stack() { FloatingPlayerComponent() } }
}
```

1.5 初始化

复制代码

```
import UIAbility from '@ohos.app.ability.UIAbility'
import window from '@ohos.window'
import { XMEcoSDK, FloatingPlayerManager } from '@xmly/xmecosdk'

export default class EntryAbility extends UIAbility {
    onCreate(): void { XMEcoSDK.getInstance().initialize(this.context) }

    onWindowStageCreate(windowStage: window.WindowStage): void {
        const fpMgr = FloatingPlayerManager.getInstance()
        fpMgr.bindWindowStage(windowStage)
        // 用户点击悬浮窗时跳转到接入方自己的播放器容器页（此处
        pages/YourPlayerPage 为例）
        fpMgr.onNavigateToPlayer = (): void => {
```

```

windowStage.getMainWindowSync().getUIContext().getRouter().pushUrl({
    url: 'pages/YourPlayerPage' })
}
XMecoSDK.getInstance().startSession(this.context)
windowStage.loadContent('pages/Index')
}

onWindowStageDestroy(): void {
    FloatingPlayerManager.getInstance().destroy() }
onDestroy(): void { XMecoSDK.getInstance().dispose() }
}

```

日志开关（默认关闭，需要时显式开启）：鸿蒙端默认 WARN（仅告警/错误）。联调需要全量

日志时，在 `initialize` 之前调用（幂等，可按构建产物条件调用）：

复制代码

```

// 方式一（推荐）：XMecoSDK 门面——同时控制 ArkTS 层热路径日志
// （dispatchNativeCall/
// HTTP 请求/切歌 URL/签名耗时）与共享层 js bundle 日志级别
XMecoSDK.getInstance().setDebugLogging(true)

// 方式二：直接调 bundle 导出（仅控制共享层 Kotlin 日志级别）
import { setDebugLogging } from '@xmly/xmecsdk'
setDebugLogging(true) // DEBUG 全量日志；不调用则为 WARN 上线安全

```

默认关闭（上线安全）。开启后热路径日志可能包含 URL 签名 token 等敏感信息，生产环境务必保持关闭。

2. WebView 集成

复制代码

```

import { XMecoSDK, getH5BaseUrl, UI_KEY_CLOSE_WEBVIEW, JSBridge } from
'@xmly/xmecsdk'
import common from '@ohos.app.ability.common'

@Component
struct WebViewDemo {

```

```

@StorageLink(UI_KEY_CLOSE_WEBVIEW) @Watch('onClose') closeFlag:
boolean = false
private controller: web_webview.WebviewController = new
web_webview.WebviewController()

onClose(): void {
    if (this.closeFlag) { this.closeFlag = false;
this.getUIContext().getRouter().back() }
}

build() {
    Web({ src: getH5BaseUrl(), controller: this.controller })
    .onControllerAttached(() => {
XMEcoSDK.getInstance().bridge.setController(this.controller) })
    // 支付 scheme 拦截 (weixin://、alipays:// 等) 由 SDK
JSBridge.handleUrl 内聚处理
    // WebResourceRequest 是 ohos web 组件内置全局类型, 无需
import
    .onOverrideUrlLoading((req: WebResourceRequest) => {
        return JSBridge.handleUrl(req.getRequestUrl(),
getContext(this) as common.UIAbilityContext)
    })
    .onPageEnd(() => {
XMEcoSDK.getInstance().bridge.injectPolyfill() })
}

```

2.1 容器标题栏适配 (URL 判断)

SDK 内的 H5 页面（播放器、专辑详情、权益等）自带标题栏；外部页面（支付中间页等）不带标题栏且无 bridge 适配。容器在页面加载时按 URL 判断：URL 含 `H5_BASE_URL` 的为 SDK 页面 → 隐藏原生标题栏（H5 自带）；其余为外部页面 → 显示原生标题栏。纯容器行为，H5 侧零适配。

复制代码

```

// Web 组件 onPageBegin 里按 URL 粗判
@State showTitleBar: boolean = false

.onPageBegin(() => {
    try {
        const url = this.controller.getUrl() ?? ''
        this.showTitleBar = !url.startsWith(getH5BaseUrl())
    }
}

```

```

    } catch (e) {
      this.showTitleBar = true
    }
  })

  build() {
    Column() {
      if (this.showTitleBar) {
        Row() { /* 返回按钮 + 标题 */ }.height(56)
      }
      Web({ src: this.url, controller: this.controller }) /* ... */
    }
  }
}

```

返回按钮建议：优先 `controller.accessBackward() ? backward() : router.back()`。

3. SDK 接口

3.1 播放控制

通过 `XMEcoSDK.getInstance().player` 访问：

 复制代码

```

const player = XMEcoSDK.getInstance().player

// 播放指定专辑曲目（breakSecond 断点秒 / listenPackageId 聆听包 ID 可选）
XMEcoSDK.getInstance().playContent('13396678', '72982155', 0, '0')

player.play(); player.pause(); player.stop(); player.toggle()
player.seekTo(30000) // 毫秒
player.setSpeed(1.5) // 倍速 0.5-3.0
player.setVolume(0.8)
player.getPlaybackMode() // 0=顺序,1=随机,2=单曲循环,3=列表循环,4=倒序

// 定时停止/睡眠定时（sleepStyle 0=无 1=自定义时间 2=播完当前 3=播完N集；
// second 1-7200; episodes 1-10）
XMEcoSDK.getInstance().playerTimedStop(1, 1800) // 30 分钟后停止
XMEcoSDK.getInstance().playerTimedStop(0) // 关闭定时
    
```

3.2 外部播放委托（渠道禁用 native 播放时）

渠道 `nativePlaybackAllowed=false`（如 noetixrobotics）时，SDK 不执行本地播放。H5 页面里用户点播放会被 SDK 拦截，默认只发错误事件（接入方收不到播放意图）。

若接入方需要用自己的播放器播放，注册 `setExternalPlaybackProviderAsync`，SDK 会在拦截时把 H5 传入的播放参数以 JSON 字符串回调给接入方：

复制代码

```
setExternalPlaybackProviderAsync((json: string) => {
    // json 字段:
    //  albumId      string    专辑 ID（必填）
    //  trackId      string    曲目 ID（H5 只指定专辑时为空串）
    //  trackTitle   string    曲目标题（可能为空串）
    //  duration     number    曲目时长，秒（可选）
    //  breakSecond  number    断点续播位置，秒（可选）
    const req = JSON.parse(json)
    // 在此用自己的播放器播放，例如：
    myPlayer.play({
        albumId: req.albumId,
        trackId: req.trackId,
        title: req.trackTitle,
        seekToMs: (req.breakSecond ?? 0) * 1000,
    })
})
```

说明：

- 回调在 SDK 拦截 native 播放后立即触发，接入方可在其中启动自己的播放器。
- 参数以 JSON 字符串传递（鸿蒙 ArkTS 跨边界限制），接入方 `JSON.parse` 即可。
- 封面（cover_url）不在回调参数内——拦截发生在 SDK 拉取专辑详情之前，此时无封面。如需封面，接入方可用 `albumId` 自行查询。
- 未注册时维持原行为（H5 播放被拦截，接入方收不到意图）。

高级播放设置

复制代码

```
player.setAlbumSort(albumId, true) // 正序
player.setSkipPlay(albumId, 30, 15) // 跳头30s跳尾15s
player.setPlaybackSpeed(1.5, albumId, true, true) // 专辑级倍速
const speedJson: string = player.getPlaybackSpeed(albumId) // JSON:
{albumId,rate,useCurrentAlbum}
```

```
const skipJson: string = player.getSkipPlay(albumId) // JSON:
{albumId: skipBeginning, skipEnding: }
```

状态查询

[复制代码](#)

```
player.isPlaying() // boolean
player.getCurrentTrack() // JSON string
(id/title/artist/cover_url/duration/album_id...) 或 null
player.getPlaylist() // JSON string (数组)
player.getCurrentPositionMs() // 当前位置 (毫秒)
player.getDurationMs() // 总时长 (毫秒)
```

事件监听

[复制代码](#)

```
player.addPlayerListener({
  onPlaybackStarted: (track: PlayableTrack | null) => {},
  onPlaybackPaused: (track: PlayableTrack | null) => {},
  onPlaybackCompleted: () => {},
  onPlaybackError: (msg: string, code: number, track: PlayableTrack |
null) => {},
  onPlaybackProgressUpdate: (pos: number, dur: number, pct: number)
=> {}, // 毫秒
// 共 15 个可选回调，只实现需要的
})
```

3.2 播控门控

[复制代码](#)

```
XMEcoSDK.getInstance().setNativePlaybackAllowed(false) // 禁用 (false 时
暂停+2600)
```

音频焦点

复制代码

```
XMEcoSDK.getInstance().requestAudioFocus()    // 返回 boolean
XMEcoSDK.getInstance().abandonAudioFocus()    // 返回 boolean
XMEcoSDK.getInstance().setAudioFocusHeld(true) // true=请求, false=释放,
返回 boolean
```

3.3 登录

通过异步回调获取结果（callback 收 JSON string）：

复制代码

```
const sdk = XMEcoSDK.getInstance()

sdk.checkLoginStatus((result: string, error: string | null) => {
    // result JSON: {isLogin, loginType, hasValidToken, hasUserProfile,
    accessToken}
})

sdk.getUserNickname((result: string, error: string | null) => { /*
{nickname} */ })
sdk.getUserAvatarUrl((result: string, error: string | null) => { /*
{avatarUrl} */ })
sdk.getUserId((result: string, error: string | null) => { /* {uid} */ })
sdk.loginWithTicket('ticket_xxx', (result: string, error: string |
null) => {
    // result JSON: {success, loginType, message, errorCode}
})
// 三方 token 登录: infoJson 为服务端约定的三方信息 JSON (如
{"ticket": "..."}), 透传为 third_info
sdk.loginWithThirdParty('{"ticket": "ticket_xxx"}', (result: string,
error: string | null) => { /* LoginResult JSON */ })

// 登录状态监听（单一监听器语义，后注册覆盖先注册）：登录成功/失败/登出时回调
sdk.addLoginStatusListener({
    onLoginSuccess: (loginStatusJson: string): void => { /*
{isLogin, loginType, ...} */ },
    onLoginFailed: (error: string, errorCode: string): void => { /* 失败
原因 + ExternalLoginError 枚举名 */ },
    onLogout: (): void => { /* 登出 (含 token 失效) */ },
})
sdk.removeLoginStatusListener()
```



```

// 自动确保已登录：已登录直接回调成功；未登录取 deviceInfoJson 走三方登录
sdk.ensureLoginWithThirdPartyAuth('{ "device_id": "...", "ticket": "..."}',
(result: string, error: string | null) => { /* LoginResult JSON */ })

// 刷新 native 登录态（H5 侧拿到新 token 后回写 native; expiresIn 毫秒）
sdk.refreshNativeAuthEntity('access_token', 'refresh_token', 86400000,
'541487411', (result: string, error: string | null) => { /* LoginResult
JSON */ })

// 登录事件上报
sdk.reportLogin('device_sn', (result: string, error: string | null) =>
{ /* {"success":true} */ })

// 便捷查询
sdk.isUserLogin((result: string, error: string | null) => { /*
{"isLogin":bool} */ })
sdk.isVipUser((result: string, error: string | null) => { /*
{"isVip":bool} */ })

// 登录并按权益状态决定是否建立登录态（耦合登录与权益领取）
// 适用场景：接入方希望在登录时校验是否有生态权益，无权益则不建立登录态
sdk.loginAndClaimIfNeeded(
    '{"ticket":"ticket_xxx"}', // loginInfoJson 登录凭据，同
loginWithThirdParty 的 infoJson
    // ticket （此处ticket是手机号加密后的登录
    票据）
    null, // vipDeviceInfo 权益查询设备信息，为空时回
    退 VipTicketProvider.getVipDeviceInfo()
    // 含 device_id + ticket （此处ticket是设备
    SN加密后的票据）
    (result: string, error: string | null) => {
        // result JSON: {loginResult:{success,...}, benefitQuery:
        {status,claimed,claimSuccess,message}|null}
        // loginResult.success → 是否已建立/保持登录态
        // benefitQuery.status → 权益状态（0..4），null=未查到/查询失败
        // benefitQuery.claimed → 是否触发领取（仅 status==4）
        // benefitQuery.claimSuccess → 领取结果（claimed=true 时有意义）
    }
)

```

```

sdk.logout((result: string, error: string | null) => { /* {success} */
})

// 登录成功数据透传（实际登录成功时触发，影子登录除外）
// 覆盖：SDK 主动登录
（loginWithTicket/loginWithThirdParty/loginAndClaimIfNeeded 转正）+ H5 同步登录（updateAuthEntity）
// 影子登录（shadowLogin，仅查权益的临时 token，不落库）不触发
sdk.setLoginDataListener((loginDataJson: string): void => {
    // loginDataJson:
    {"access_token":"...", "refresh_token":"...", "uid":3192, "login_type":"THIRD_PARTY"}
    // login_type: TICKET / MOBILE / THIRD_PARTY（区分登录来源，便于埋点）
})

```

3.4 权益注入

注册 VIP 票据提供者，SDK 在权益查询/领取时委托回调。

各方法说明：

方法	必填	返回类型	说明
<code>getVipTicket</code>	✓	<code>string</code>	VIP 票据（JWT 字符串），未注入时 H5 调用会抛 Error
<code>getVipDeviceInfo</code>	○	<code>string</code>	本机设备信息 JSON 字符串 ，默认 <code>'{}'</code>
<code>getThirdDeviceInfo</code>	○	<code>string</code>	三方设备信息 JSON 字符串 ，默认 <code>'{}'</code>


```

class MyVipTicketProvider implements VipTicketProvider {

    // 【必填】VIP 票据（此处ticket是设备sn加密）。直接返回裸字符串，勿包 JSON
    // （SDK 会自动包 {'ticket': ...} 发给 H5）
    // 同步方法无法 await 网络：需在外部提前异步刷新票据并缓存，此处只读缓存
    getVipTicket(): string {
        return AppPrefs.vipTicket ??
        'eyJhbGciOiJIUzI1NiJ9.eyJ1aWQiOiJmOTJ9.xxxxxx'
    }

    // 【可选】本机设备信息（此处ticket是设备sn加密），返回 JSON 字符串；无需此
    // 信息返回 '{}'
    getVipDeviceInfo(): string {
        const info: Record<string, string> = {
            'device_id': DeviceUtils.deviceId,
            'ticket': AppPrefs.vipTicket ?? '',
        }
        return JSON.stringify(info)
        // 输出：
        {"device_id":"DILZUDID5504ADYC257M0010","ticket":"eyJ0eXA..." }

    // 【可选】三方设备信息（此处ticket是手机号加密），返回 JSON 字符串；无三方
    // 信息返回 '{}'
    getThirdDeviceInfo(): string {
        const ticket = ThirdParty.ticket
        if (!ticket) return '{}'
        const info: Record<string, string> = {
            'device_id': ThirdParty.deviceId,
            'ticket': ticket,
            'model': deviceInfo.model,
        }
        return JSON.stringify(info)
    }

    // 【必填】权益领取结果回调。result 格式：{"success":true,"msg":"领取成功"}
    onVipClaimResult(result: string): void {
        try {
            const json: Record<string, Object> = JSON.parse(result)

```

```

        console.info(`[Vip] 领取结果: success=${json['success']} ?? false`,
msg=${json['msg']} ?? ''`)
    } catch (e) {
        console.error(`[Vip] 解析领取结果失败: ${result}`)
    }
}
}

// 注册（必须先初始化 SDK；页面 aboutToAppear 注入，对齐 iOS viewDidLoad /
Android onCreate）
XMEcoSDK.getInstance().setVipTicketProvider(new MyVipTicketProvider())

// 取消注册（退出登录时）

```

💡 **时序：**必须 SDK 初始化之后再注入；建议在进入权益页的 `aboutToAppear` 注入，不要等用户点击才注入。

3.5 数据获取

通过异步回调获取结果（callback 收 JSON string）：

✕ 复制代码

```

sdk.fetchCommonSubscribeList(1, 10, (result: string, error: string |
null) => {
    // result JSON: [{album_id, title, cover_url, track_count, ...}]
})
sdk.subscribeAlbum(albumId, (result, error) => { /* {success} */ })
sdk.unsubscribeAlbum(albumId, (result, error) => { /* {success} */ })
sdk.isSubscribeBatch('1,2,3', (result, error) => { /* {albumId:
boolean} */ })
sdk.fetchCommonHistoryList(1, 10, (result, error) => { /* [{track_id,
title, ...}] */ })
sdk.fetchListenList(1, 10, '1,2', (result, error) => { /* {groupId:
{title,album_ids}} */ })
sdk.fetchAlbumBatch('1,2,3', (result, error) => { /* [{album_id,...}]
*/ })
sdk.textAiSearch('三国', '10', (result, error) => { /* {albums:[...],
total} */ })
sdk.fetchEcoBenefitQualification(null, false, (result, error) => {
    // 生态权益数据 (result JSON: {ecological_benefit_status,
benefit_type, ...})
}

```

```

// thirdDeviceInfo 为空时自动回退取
VipTicketProvider.getVipDeviceInfo, 并自动带 uid
// autoClaim=true 时查询到可领权益 (status==4) 自动领取, 结果回调
onVipClaimResult
// 权益判断: ecological_benefit_status == 4 (HAS_BENEFIT) 表示有权益可
领取
})
sdk.fetchDeviceVipInfo((result, error) => {
// 设备 VIP 信息 (result JSON: {is_vip, subscribe_activity,
expire_at})
// subscribe_activity 为权益领取页 URL
})
sdk.verifyAlbumPermission(albumId, (result, error) => { /*

```

`fetchEcoBenefitQualification` 参数说明:

参数	类型	默认值	说明
----	----	-----	----

| `thirdDeviceInfo` | `string \ | null` | `null` | 三方设备信息 **JSON 字符串** (如 `'{"device_id": "xxx"}'`)。为空时 SDK 自动从 `VipTicketProvider` 获取 |

| `autoClaim` | `boolean` | `false` | `true` 时查询到可领权益 (`status==4`) 自动领取, 结果回调 `onVipClaimResult`; `false` 时仅查询 |

3.5.1 登录并领取权益 (loginAndClaimIfNeeded)

`loginAndClaimIfNeeded` 把"登录"和"权益领取"耦合为原子操作, 按当前登录态分两条路径:

未登录路径 (核心场景):

1. 影子登录: 拿到 token + uid, 不落库、不通知监听器、不切状态机 (临时态, 仅用于查权益)
2. 用影子 token 查询权益资格:
 - 查询技术失败 → 不登录, 返回失败
 - `status==4` (有权益) → 正式登录 + 自动领取
 - `status==0` (无权益) → 不建立登录态, 返回失败
 - 其他状态 (1/2/3) → 正式登录

已登录路径 (用户已登录后再调用):

1. 跳过影子登录, 用现有 token 查询权益 (不覆盖当前登录态), 每次调用都回调 `setLoginDataListener`
2. `status==4` → 自动领取, 保持登录态

3. 其他状态 → 直接放行（不退登）

⚠ **并发安全**：`loginAndClaimIfNeeded` 内部用 `Mutex` 串行化，并发调用会排队执行。SDK 为单全局登录态，多设备连续调用时第二次走已登录路径用的是第一次设备的 `token`。多设备切换建议先 `logout()` 再调本方法。

💡 **领取结果双通道**：领取结果既回调 `VipTicketProvider.onVipClaimResult`（与 `fetchEcoBenefitQualification` 一致），也通过返回 JSON 的 `benefitQuery.claimSuccess` 字段提供。

callback 收到 JSON 结构：`{loginResult:{success,loginType,message,...},benefitQuery:{status,claimed,claimSuccess,message}|null}`

字段	类型	说明
<code>loginResult.success</code>	<code>boolean</code>	是否已建立/保持登录态

| `benefitQuery` | `object \ | null` | `null` =未进入查询/查询技术失败 |

<code>benefitQuery.status</code>	<code>number</code>	权益状态 0..4
----------------------------------	---------------------	-----------

| `benefitQuery.claimSuccess` | `boolean \ | null` | 领取结果；未领取时为 `null` |

参数说明：

参数	类型	说明
<code>loginInfoJson</code>	<code>string</code>	登录凭据 JSON，透传为登录接口的 <code>third_info</code> 。含 <code>ticket</code> （手机号加密后的登录票据），如 <code>'{"ticket":"eyJ0eXA..."}'</code>

| `vipDeviceInfo` | `string \ | null` | 权益查询设备信息 JSON，为空时回退 `VipTicketProvider.getVipDeviceInfo()`。含 `device_id` + `ticket`（设备 SN 加密后的 VIP 票据），如 `'{"device_id":"xxx","ticket":"eyJ0eXA..."}'` |

⚠ **两个 ticket 含义不同**：`loginInfoJson` 的 `ticket` 是手机号加密的登录票据（用户维度）；`vipDeviceInfo` 的 `ticket` 是设备 SN 加密的 VIP 票据（设备维度，与 `getVipTicket()` 同源）。

3.6 悬浮播放器

[复制代码](#)

```
const mgr = FloatingPlayerManager.getInstance()
mgr.show(); mgr.hide(); mgr.isShowing(); mgr.togglePlay()
```

3.7 页面跳转 / 埋点

每条音频播放结束（切换/暂停/完成/列表完成）时回调本次累计播放时长：

[复制代码](#)

```
sdk.addCommonActionListener({
  onCommonAction: (action: string, albumId: string, trackId: string |
null) => {
    // action: "openPlayer" | "openAlbumDetail"
  }
})
sdk.addAnalyticsListener({
  onPlayRecord: (event: PlayRecordEvent): void => {
    // event.albumId / trackId / playSeconds（本次净播放秒数，已扣除暂停）/ startAt
  }
})
```

3.8 支付

[复制代码](#)

```
const sdk = XMEcoSDK.getInstance()
sdk.getPaymentType((result, error) => { /* {paymentType} (0=SDK,1=HOST) */ })
sdk.setPaymentType(1, (result, error) => { /* {success} (1=接入方支付) */ })

// 他方支付监听（paymentType=HOST 时，H5 触发支付时回调接入方拉起自有支付）
sdk.addHostPaymentListener({
  onHostPaymentRequired: (orderId: string | null, amount: string |
null,
                          productId: string | null, paramsJson:
string | null): void => {
```

```
// 拉起接入方自有支付，完成后调 notifyHostPaymentResult 回灌结果
})

// 回灌支付结果（code: 0=成功，非0=失败）
sdk.notifyHostPaymentResult(0, '成功', null, (result, error) => { /*
{success} */ })

// 支付结果监听（接收支付完成通知）
sdk.addPaymentListener({
  onPaymentResult: (code: number, message: string, dataJson: string |
null): void => {}
})
```

3.9 渠道信息



复制代码

```
sdk.getChannelName(); sdk.getAppKey(); sdk.getSN(); sdk.getChannelId()
sdk.isPlayerEnabled(); sdk.isEcoLinkModuleAvailable()
sdk.isAIModuleAvailable(); sdk.isAdModuleAvailable()
```

3.10 URL 构建



复制代码

```
sdk.buildDefaultPlayerUrl('13396678', '72982155', 0) // H5 播放页 URL
sdk.buildAlbumDetailUrl('13396678') // H5 专辑详情 URL
```

4. 错误码

码	说明	处理建议
1000	未知错误	检查参数和网络
1001	网络错误	检查网络连接
2000	播放错误	检查音频资源有效性
2019	音频列表为空	该专辑暂无可播放内容

2026	播放鉴权错误	检查用户权限/登录状态
2501	非版权内容	该内容暂时无法播放
2600	原生播控被禁用	<code>setNativePlaybackAllowed(true)</code> 后重试
3000	登录失败	检查登录凭据
3001	令牌已过期	重新登录
3006	用户未登录	引导用户登录

5. 注意事项

1. 薄壳页面：宿主必须提供 `pages/FloatingPlayer` @Entry 页面并注册到 `main_pages.json`。
2. 初始化：在 `UIAbility` `onCreate` 调用 `initialize(this.context)`，幂等。
3. 版本号：SDK 版本 `2.0.0` 不可修改。
4. 权益：`setVipTicketProvider` 的 `getVipTicket` / `onVipClaimResult` 必填；`getVipDeviceInfo` / `getThirdDeviceInfo` 可选（返回 JSON 字符串，默认 `'{}'`）。
5. 数据获取回调：所有 `callback` 为异步触发，`error` 非 `null` 表示失败。
6. SDK 生命周期与登录态管理（重要）：
 - 退出登录只需调 `XMEcoSDK.getInstance().logout(...)`（或 H5 调 `AuthBridge.logout()`），清除 `token` + 通知监听器。
 - 关闭 H5 页面只需销毁 `WebView` 容器。
 - `XMEcoSDK.getInstance().release()` 是 SDK 彻底销毁（销毁播放器、关闭网络引擎、不可逆），仅在 App 完全退出 SDK 功能模块时调用。
 - `VipTicketProvider` 的方法是同步的（ArkTS 桥接），必须同步返回结果（提前异步刷新票据并缓存，方法内只读缓存）。